

CSE 543: Computer Security

Fall 2025

Project 1: SSH Protocol and MITM Attack

Due: 11:59 pm (Eastern time), September 22, 2025

September 8, 2025

1 Introduction

This project has two main parts. In the first part, you will develop a client-server system that provides secure file transfer. You will implement the SSH protocol to construct a secure channel between the client and server over which to perform secure file transfer. You will use the OpenSSL library to implement the SSH protocol. The SSH protocol produces a symmetric key shared between the client and server, and you will use the OpenSSL library once again to use that key to transfer the file. In the second part, you will develop a man-in-the-middle (MITM) attack against the SSH protocol that you implemented in part 1. You will create a MITM that pretends to be the target server but instead opens a secure connection to the client that it can use to compromise the integrity of the communication.

The system you produce must run on the Westgate Linux lab machines. These machines are named e5-cse-135-XX.cse.psu.edu, where XX is a number between 01 and about 40. All these machines should be identical and already have the OpenSSL library installed. You should SSH into those machines to verify that your code works. We developed and tested the project code on those machines, so should work fine, but it is up to you to make sure. If you are working from outside of the PSU wireless, use may need to use a VPN which you can download from here (<https://agsci.psu.edu/it/how-to/topics/remote-access-network-and-mobile-devices/globalprotect-vpn>). **You will need to speak to the CSE IT folks (helpdesk@cse.psu.edu) if you do not have access to those machines.**

2 Overview

2.1 Part 1

The main task in part 1 is to implement the SSH protocol as described in the paper (SSH.pdf available on Canvas). Since any user is authorized to upload a file in this project, you only have to implement Steps 1-4 of the protocol, corresponding to messages 1-4 in the ProtoMessageType enumeration in `cse543-proto.h`. You are going to use the OpenSSL functions provided to implement the SSH protocol.

2.2 Part 2

The original SSH protocol that you will implement in part 1 is prone to a kind of MITM attack called **Server Spoofing**. In this attack, a malicious entity may intercept client communications intended for a target server to create a MITM connection, where the malicious entity pretends it is the server to

the client and as a client to the target server. Since the SSH protocol does not authenticate that the public key provided by the server is really associated with the target server to which the client intends to connect, a malicious entity can take advantage of this. The SSH systems take some measures to prevent this attack outside the crypto protocol. You will extend the server of part 1 to produce a MITM server that performs this server spoofing MITM attack against the (unmodified) client and (mostly unmodified) server implemented in part 1.

3 Project Tasks

You should first download the zip files. The files have the initial code for part 1 and part 2. The project includes source code and a Makefile required for compiling the files. We suggest you perform the project tasks listed below in the following order.

3.1 Part 1

The initial version of the program includes two functions `test_rsa` and `test_aes` that encrypt a message. These two functions demonstrate how to perform symmetric and public key encryption with the OpenSSL library, which should be a big help in the project. Note they won't work correctly until you have some degree of randomness in the `generate_pseudorandom_bytes` function.

3.1.1 Write the functions to build encrypted messages for sending and decrypted received messages

There are two pairs of functions for you to implement: `seal_symmetric_key/unseal_symmetric_key` for public key crypto and `encrypt_message/decrypt_message`. These encryption functions must perform encryption and produce buffers containing the data necessary for the other party to decrypt. The decryption functions must extract the necessary information from a sent buffer and perform the decryption.

3.1.2 Write the function to generate pseudo-random values

Develop the function `generate_pseudorandom_bytes` by using the OpenSSL functions for producing pseudo-random values.

3.1.3 Develop the SSH Protocol

Implement the client and server portions of the SSH protocol, as described in the paper (SSH.pdf available on Canvas). There are two functions `client_authenticate` and `server_protocol` to be implemented.

3.1.4 Implement Secure File Transfer

Implement `client_secure_transfer` to connect to the server, perform authentication and establish session key, securely transfer the specified file, close the connection, and return success/failure.

3.2 Part 2

Use the client and server you implemented in part 1. For this attack, you will run these components unmodified except for a small modification to one function (`receive_file`) used by the server.

3.2.1 Implement server spoofing attack

In the MITM attack, you will modify the code from part 2 (see below) to perform the server spoofing attack.

4 Implementation

4.1 Part 1

4.1.1 Create public and private keys

You can create the public and private keys using the OpenSSL system using the following commands:

```
# generate key pair - mykey.pem holds private key
openssl genrsa -out mykey.pem 2048

# extract public key in basic format - pubkey.pem is in PKCS#8 format
openssl rsa -in mykey.pem -pubout -out pubkey.pem

# convert public key to RSA format - rsapub.pem holds public key
openssl rsa -pubin -in pubkey.pem -RSAPublicKey_out > rsapub.pem
```

4.1.2 Running the Client and Server

The server program will be started by the following command line:

```
cse543-p1-server <private-key-file> <public-key-file>
```

where (1) the <private-key-file> is the name of the file that stores the private key for the server and (2) <public-key-file> is the name of the file that stores the corresponding rsa format public key for the server.

The client program will be started by the following command line:

```
cse543-p1 <file-to-transfer> <server-ip-address>
```

where (1) the <file-to-transfer> is the file path name of the file to transfer from the client to the server and (2) <server-ip-address> is the IP address of the server host.

Start the server first, as it will wait for connection requests from clients. When a connection request is received from a client the sequence of steps will be performed.

4.1.3 Perform the SSH protocol

The client will initiate the SSH protocol to produce a symmetric key to be shared by the client and server.

4.1.4 Transfer the file

The <file-to-transfer> will be sent encrypted and integrity-protected from the client to the server. The server will store the file in a directory called “shared” under the directory from which the server is run.

4.1.5 Server awaits next request

The client will terminate and the server will await the next request from the next client.

4.2 Part 2

The code for part 2 consists of one new file `cse543-p2.c`, which replaces the `cse543-p1.c` file from part 1. The `cse543-p2.c` code only differs from the part 1 code only that the interface to start the server is slightly different (see MITM USAGE in the file). The first two arguments are the MITMs public and private keys as before. These keys will be different than the target server's public and private keys, of course. The third argument is still an IP address, but it is the IP address of the real (target) server. The fourth argument is the "MITM option" which you should set to 1 to perform the server spoofing attack. In summary, MITM should run as below:

On machine 1:

```
cse543-p1 <filename> <IP-address-of-machine-1>
```

```
cse543-p2-server <private_key_file> <public_key_file> <IP-address-of-machine-2> 1
```

On machine 2:

```
cse543-p1-server <private_key_file> <public_key_file>
```

4.2.1 Use Your Part 1 Code

The client and target server will be largely unchanged from part 1. You will run MITM with option "1". This will invoke `server_secure_transfer` from part 1. You need to modify that function to perform the MITM proxy attack, as described below.

4.2.2 Server Spoofing Attack

In this attack, you will modify the code in server secure transfer to perform the SSH protocol as the server to a part 1 client, but also initiate a new SSH connection to a part 1 server as the client. This attack emulates the case where a MITM may obtain an IP address for another server or sit between the client and server on the network. The attack must: (1) construct a SSH connection with the client sufficient for the client to transfer the file (from part 1); (2) replay the client messages to create a second SSH connection to another part 1 server; and (3) append the text **"YOUR TRANSFER IS UNDER RISK"** to the client's file before sending it to the server. This shows that the integrity of the initial file is in jeopardy. **To test for this, run the client and MITM server on the same machine, and have the MITM server communicate with the target server on another machine.** Both servers will receive connections listening to the same port (9165). [Hint: You might need to make a small change to the `receive_file` function.]

5 Deliverable

Please submit a zip containing only two files:- `part1/cse543-proto.c` and `part2/cse543-proto.c`.

6 Grading (Total 100 points)

1. We can build and run what you have submitted without any compilation or runtime errors (10 points).
2. Encrypted communication works (10 points).
3. Generate pseudo-random data (10 points).
4. SSH protocol (35 points).

5. Transfer file securely (10 points).
6. Server spoofing attack works (25 points).